

Vite Projesini Dockerize Etme

Bu doküman, Vite ile geliştirilmiş bir React projesinin Docker container'ı içinde nasıl çalıştırılacağını ve production ortamına nasıl deploy edileceğini adım adım anlatır.

İçindekiler

- [1. Docker Nedir?](#)
- [2. Neden Dockerize Edelim?](#)
- [3. Ön Gereksinimler](#)
- [4. Multi-Stage Build Yaklaşımı](#)
- [5. Dockerfile Oluşturma](#)
- [6. Nginx Konfigürasyonu](#)
- [7. .dockerignore Dosyası](#)
- [8. Build ve Run Komutları](#)
- [9. Docker Compose ile Çalıştırma](#)
- [10. Geliştirme Ortamı \(Dev Mode\)](#)
- [11. Environment Variables \(Ortam Değişkenleri\)](#)
- [12. Image Boyutunu Optimize Etme](#)
- [13. Sık Karşılaşılan Hatalar](#)

Docker Nedir?

Docker, uygulamaları **container** adı verilen izole birimler içinde paketleyip çalıştırmayı sağlayan bir platformdur. "Benim makinemde çalışıyor ama..." problemini ortadan kaldırır.

- Image:** Uygulamanın tüm bağımlılıkları ile birlikte paketlenmiş halidir (şablon).
- Container:** Image'ın çalışan bir kopyasıdır (instance).
- Dockerfile:** Image'ın nasıl oluşturulacağını tarif eden dosyadır.

Neden Dockerize Edelim?

- Tutarlılık:** Geliştirme, test ve production ortamı aynı olur.
- Hızlı deployment:** Image'ı her yere push edebilirsiniz (AWS, GCP, DigitalOcean).
- İzolasyon:** Host makineyi kirletmez; Node sürümü, kütüphane çakışması olmaz.
- Ölçeklenebilirlik:** Kubernetes/Docker Swarm ile kolayca scale edilir.

- **CI/CD:** Otomatik pipeline'larda kolayca kullanılır.

Ön Gereksinimler

```
# Docker'ı kontrol edin
docker --version
docker compose version

# Mac için Docker Desktop kurulumu:
# https://www.docker.com/products/docker-desktop
```

Multi-Stage Build Yaklaşımı

Vite, **statik dosyalar** üretir (HTML, CSS, JS). Bunun için iki aşamalı bir yapı kullanırız:

1. **Build aşaması:** Node.js image'ı içinde `npm run build` çalıştırılır. Tüm `node_modules`, TypeScript derleyici vs. burada kalır.
2. **Serve aşaması:** Sadece `dist/` klasörü hafif bir **Nginx** image'ına kopyalanır.

Bu yaklaşım sayesinde final image boyutu **1.2 GB → 25 MB** seviyesine düşer.

Dockerfile Oluşturma

Proje kök dizinine (`dokuman-paneli/`) aşağıdaki `Dockerfile` dosyasını oluşturun:

```
# =====
# Stage 1: Build
# =====
FROM node:22-alpine AS builder

# Çalışma dizinini ayarla
WORKDIR /app

# Önce sadece package dosyalarını kopyala (cache optimizasyonu)
COPY package.json package-lock.json ./

# Bağımlılıkları yükle (ci komutu daha hızlı ve deterministik)
RUN npm ci

# Tüm kaynak kodunu kopyala
COPY . .

# Production build al
RUN npm run build

# =====
# Stage 2: Serve (Nginx)
# =====
FROM nginx:alpine AS production

# Default nginx config'i sil
RUN rm /etc/nginx/conf.d/default.conf

# Custom nginx config'i kopyala
COPY nginx.conf /etc/nginx/conf.d/default.conf

# Build edilmiş dosyaları nginx'in serve ettiği dizine kopyala
COPY --from=builder /app/dist /usr/share/nginx/html

# 80 portunu expose et
EXPOSE 80

# Nginx'i foreground'da çalıştır
CMD ["nginx", "-g", "daemon off;"]
```

Önemli Noktalar

- **node:22-alpine** → Alpine Linux tabanlı, çok küçük boyutlu Node image'ı.
- **COPY package*.json önce yapılır** → Sadece bağımlılıklar değişmedikçe Docker cache'i kullanılır, build hızlanır.
- **npm ci** → **package-lock.json** 'a göre temiz kurulum yapar, **npm install** 'dan hızlıdır.
- **AS builder** → Stage'e isim verir; ikinci stage'de **--from=builder** ile dosya alabiliriz.

Nginx Konfigürasyonu

React Router gibi **client-side routing** kullanan SPA'larda sayfayı yenilediğinizde "404 Not Found" hatası alırsınız. Bunu çözmek için tüm istekleri `index.html` 'e yönlendirmemiz gerekir.

`nginx.conf` dosyasını proje köküne ekleyin:

```
server {
    listen 80;
    server_name localhost;
    root /usr/share/nginx/html;
    index index.html;

    # Gzip sıkıştırma
    gzip on;
    gzip_vary on;
    gzip_min_length 1024;
    gzip_types text/plain text/css text/xml text/javascript
        application/javascript application/json application/xml+rss;

    # Statik asset'leri uzun süre cache'le
    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    # SPA fallback: tüm route'ları index.html'e yönlendir
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
}
```

.dockerignore Dosyası

`node_modules` ve gereksiz dosyaları image'a kopyalamamak için `.dockerignore` oluşturun:

```
node_modules
dist
.git
.gitignore
.env
.env.local
.env.*.local
*.log
.DS_Store
.vscode
.idea
README.md
Dockerfile
docker-compose.yml
.dockerignore
coverage
.cache
```

Bu dosya olmazsa, build sırasında local `node_modules` (yüzlerce MB) image'a kopyalanır ve build süresi çok uzar.

Build ve Run Komutları

```
# Image build et (proje dizininde)
docker build -t dokuman-paneli:1.0.0 .

# Container'ı çalıştır (host 8080 -> container 80)
docker run -d -p 8080:80 --name dokuman-paneli-app dokuman-paneli:1.0.0

# Tarayıcıdan aç
open http://localhost:8080

# Logları izle
docker logs -f dokuman-paneli-app

# Container'ı durdur ve sil
docker stop dokuman-paneli-app
docker rm dokuman-paneli-app

# Image'ı sil
docker rmi dokuman-paneli:1.0.0
```

Image Boyutunu Kontrol Etme

```
docker images | grep dokuman-paneli
# dokuman-paneli 1.0.0 abc123def 2 minutes ago 25MB
```

Docker Compose ile Çalıştırma

Birden fazla servisi yönetmek veya komutları kısaltmak için `docker-compose.yml` kullanın:

```
services:
  web:
    build:
      context: .
      dockerfile: Dockerfile
    image: dokuman-paneli:latest
    container_name: dokuman-paneli-app
    ports:
      - "8080:80"
    restart: unless-stopped
    environment:
      - NODE_ENV=production
    healthcheck:
      test: ["CMD", "wget", "--spider", "-q", "http://localhost:80"]
      interval: 30s
      timeout: 10s
      retries: 3
```

Compose Komutları

```
# Build edip arka planda çalıştır
docker compose up -d --build

# Logları izle
docker compose logs -f

# Durdur ve container'ları sil
docker compose down

# Image'ları da silerek tam temizlik
docker compose down --rmi all --volumes
```

Geliştirme Ortamı (Dev Mode)

Production build yerine **Vite dev server**'ı container içinde çalıştırmak isterseniz ayrı bir `Dockerfile.dev` oluşturun:

```
FROM node:22-alpine

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci

COPY . .

EXPOSE 5173

# HMR'ın çalışması için --host 0.0.0.0 şart
CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0"]
```

`vite.config.ts` dosyanıza dev server konfigürasyonu ekleyin:

```
export default defineConfig({
  plugins: [react()],
  server: {
    host: '0.0.0.0',
    port: 5173,
    watch: {
      usePolling: true, // Docker volume'larda HMR için gerekli
    },
  },
})
```

Docker Compose dev konfigürasyonu (`docker-compose.dev.yml`):

```
services:
  web-dev:
    build:
      context: .
      dockerfile: Dockerfile.dev
    container_name: dokuman-paneli-dev
    ports:
      - "5173:5173"
    volumes:
      - ./app
      - /app/node_modules # node_modules'u host'tan ayır
    environment:
      - CHOKIDAR_USEPOLLING=true
```

Çalıştırma:

```
docker compose -f docker-compose.dev.yml up
```

Environment Variables (Ortam Değişkenleri)

Vite'ta `VITE_` öneki ile başlayan env değişkenleri **build time**'da bundle'a gömülür. Yani container çalıştırılırken `-e` ile geçemezsiniz; build sırasında geçmelisiniz.

Build-Time Env Geçirme

```
FROM node:22-alpine AS builder

ARG VITE_API_URL
ARG VITE_APP_NAME

ENV VITE_API_URL=$VITE_API_URL
ENV VITE_APP_NAME=$VITE_APP_NAME

WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build
```

Build komutu:

```
docker build \
  --build-arg VITE_API_URL=https://api.example.com \
  --build-arg VITE_APP_NAME="Doküman Paneli" \
  -t dokuman-paneli:1.0.0 .
```

Runtime Env (Gelişmiş)

Runtime'da env değiştirmek için `envsubst` veya `window.__ENV__` pattern'i kullanabilirsiniz. Detay için [bu makaleye](#) bakın.

Image Boyutunu Optimize Etme

Image	Boyut	Not
<code>node:22</code>	~1.1 GB	Tam Debian, geliştirme için ideal
<code>node:22-slim</code>	~250 MB	Daha küçük Debian
<code>node:22-alpine</code>	~180 MB	Alpine Linux, prod build için
<code>nginx:alpine</code>	~25 MB	Final image için

İpuçları:

- Multi-stage build kullanın (en önemli optimizasyon).
- `.dockerignore` dosyasını eksiksiz yazın.
- `npm ci --omit=dev` ile sadece production bağımlılıklarını yükleyin.
- Layer cache'i için `COPY package*.json` adımını mutlaka ayrı tutun.
- Mümkünse `distroless` veya `scratch` base image kullanın (ileri seviye).

Sık Karşılaşılan Hatalar

1. "Cannot find module" Hatası

Sebe: `node_modules` host'tan kopyalanmış olabilir (farklı OS için derlenmiş binary).

Çözüm: `.dockerignore` 'a `node_modules` ekleyin ve image'ı yeniden build edin.

2. Sayfa Yenilendiğinde 404 Hatası

Sebe: Nginx, React Router'ın client-side route'larını bilmiyor.

Çözüm: `nginx.conf` içinde `try_files $uri $uri/ /index.html;` satırının olduğundan emin olun.

3. HMR (Hot Reload) Çalışmıyor

Sebe: Docker volume'larda file system event'leri tetiklenmez.

Çözüm: `vite.config.ts` içinde `watch.usePolling: true` ayarını açın.

4. Build Çok Yavaş

Sebe: `node_modules` her seferinde yeniden yükleniyor.

Çözüm: `COPY package*.json` adımını `COPY . .` adımından önce yapın. Bu sayede Docker bağımlılıkları cache'ler.

5. Permission Denied Hatası

Sebe: Container içinde root olmayan user dosyalara yazamıyor.

Çözüm: Dockerfile'a `USER node` ekleyin veya dosya izinlerini düzenleyin.

6. Vite Env Değişkenleri Çalışmıyor

Sebe: Vite env'leri build time'da gömer; container çalıştırırken `-e` ile geçmek işe yaramaz.

Çözüm: `--build-arg` ile build sırasında geçin (yukarıdaki bölüme bakın).

Production Deployment İpuçları

- Image registry kullanın:** Docker Hub, GitHub Container Registry, AWS ECR veya GCP Artifact Registry.
- Tag stratejisi:** `latest` yerine semantic version kullanın (`v1.0.0` , `v1.0.1`).
- CI/CD entegrasyonu:** GitHub Actions ile push'larda otomatik build/push yapın.
- HTTPS:** Production'da Nginx'in önüne Traefik veya Caddy koyun, Let's Encrypt ile SSL alın.
- Monitoring:** Container metriklerini Prometheus + Grafana ile izleyin.
- Log aggregation:** `docker logs` yerine Loki veya ELK stack kullanın.

Özet Akış

```
[Kaynak Kod]
  ↓ docker build
[Builder Stage: Node + npm ci + vite build]
  ↓ COPY --from=builder
[Production Stage: Nginx + dist/]
  ↓ docker run
[Container: Port 80'de serve]
  ↓ docker push
[Registry: Docker Hub / ECR]
  ↓ deploy
[Production Server / Kubernetes]
```

Bu doküman ile Vite projenizi modern bir container yaklaşımıyla paketleyebilir ve herhangi bir cloud sağlayıcısında çalıştırabilirsiniz.